Critical State Detection for Adversarial Attacks in Deep Reinforcement Learning

Praveen Kumar R[†], Niranjan Kumar I[†], Sujith Sivasankaran [‡], Mohan Vamsi A [§], and Vineeth Vijayaraghavan[¶]
[†]SSN College of Engineering, Chennai, India
[‡]Sri Venkateswara College of Engineering, Chennai, India
[§]Panimalar Engineering College, Chennai, India
[¶]Solarillion Foundation, Chennai, India
{praveenkumarramesh, i.niranjankumar, sujithsivasankaran, amvamsi, vineethv}@ieee.org

***Abstract*—Deep learning plays a vital role in day-to-day applications. Recent studies show that deep learning models are not resilient against adversarial attacks, which is also applicable to Deep Reinforcement Learning (DRL) agents. Considering sensitive use cases of the DRL agents, there is a pressing need to make them robust to adversarial attacks. However, to design an efficient defense, it is imperative that we fully understand the vulnerability of such agents. In this work, we propose statistical and model-based approaches to identify critical states in an episode. Our work shows that by attacking less than 1% of the total number of states, the agent performance can be reduced by more than 40%. Furthermore, we model a *long-term impact classifier* to identify critical states. This method reduces the average compute time by 80.3% when compared to previous approaches.**

***Index Terms*—deep reinforcement learning, adversarial attack, critical point attack, real-time**

I. Introduction

Deep learning has undergone rapid advancements over the past decade. As a result, it led to breakthroughs and outstanding performances in various fields, such as image classification [1], speech recognition [2], and natural language processing [3]. Using deep neural networks, impeccable performance was achieved in image recognition tasks. This paved the way for its application in autonomous driving [4] and other crucial real-time practical scenarios.

The application of deep neural networks in reinforcement learning environments opened up new avenues [5]. As a result, deep reinforcement learning (DRL) models are employed in many practical use cases. DRL models outclassed even human professional players in a game of Go [6] and outperformed the human benchmark set in Atari 2600 games [7].

The vulnerability of deep learning models to adversarial attacks poses a massive threat because of the ever-growing importance of deep learning in practical applications [8]. Introducing an imperceptible noise [9] to the inputs perceived by the deep neural networks can reduce the performance by a large margin. The consequences can be catastrophic, especially in high-risk applications, such as self-driving cars. Misinterpreting road signs [10] and falsely predicting steering angles [11] are few examples that can produce dire consequences by simply attacking the sensory input of self-driving cars.

It is imperative to make the deep neural networks robust to adversarial attacks to ensure reliable performance in real-life applications. To design an effective defense, we need to have a good understanding of the vulnerability of the DRL agents to such attacks. Furthermore, we also need to understand the nature of the attacks that the agent might encounter. Accordingly, this can help in crafting attack specific defense. To this end, our paper makes the following contributions.

- We propose novel statistical and model-based approaches to identify the critical states in an episode. Our work shows that by attacking less than 1% of the states, the agent performance can be reduced by more than 40%.
- We model a novel *long-term impact classifier* to identify the critical states. This reduced the compute time by 80.3% when compared to the previous approaches.

The rest of the paper is structured as follows. Section II outlines the previous approaches to attack the DRL agent and their limitations. In Section III we provide the essential background information on adversarial attacks in the context of DRL. In Section IV we introduce our statistical and model-based approaches to detect critical frames. In Section V and VI, we explain the experimental setup and analyze the results obtained, respectively. Finally, we lay down our conclusions in Section VII.

II. Related Work

Deep learning models are susceptible to minute changes [9] in the input distribution. Adversarial attacks in both black box [12] and white box[13], [14] settings were investigated in context to various applications.

Adversarial attacks in the context of DRL were first studied by Huang et al. [8]. There was a significant decline in performance, even with imperceptible adversarial noise. Furthermore, attacks crafted for one task can be transferred to another task[15], [16]. Since DRL agents are now deployed in real-life crucial scenarios [17], there is a pressing need to fully understand their vulnerabilities to take counter-measures against those attacks.

Gleave et al. [18] proposed an adversarial policy to deceive the target agent in a zero-sum game. The mere presence of the adversary had a detrimental effect on the agent's performance and abruptly ended the episode. However, this method involves direct manipulation of the environment perceived by the agent.

978-1-6654-4337-1/21/$31.00 ©2021 IEEE
DOI 10.1109/ICMLA52953.2021.00279

In reality, attackers have very limited access to the environment and target agent parameters to craft the attack[19].

Instead of attacking all the states, the attacker could attack a subset of states [20]–[24] that are important in determining the outcome of the episode. Lin et al. [20] discussed the first approach to determine the time steps in which the adversary should attack the agent. This is done by calculating the range of the softmax value of the action distribution. The authors of [21] considered the difference between the Q-values estimated by the adversary and the target agent to determine if a perturbation can be crafted at that time step. Extending this work, [22] used weighted Q-values to give importance to specific actions. Sun et al.[23] analyzed the impact of performing a sequence of sub-optimal actions for certain time steps to deem if it is beneficial for the attacker to attack the current state. The authors of [24] added noise to small regions in the state representations and reduced the performance of the agent drastically.

Even though the aforementioned methods are effective in reducing the agent's performance, they tend to overestimate the number of frames to attack. Furthermore, [23] computes the quality of the state that the agent will transition to, in an adversarial setting by performing M-step simulation at each time step. Since all state transitions are computed by the CPU, it leads to computational overhead. To overcome this, we model the classifier to approximate the approach discussed in [23]. The strategy discussed in [22] is quite restricted to the task and thus, not transferable. However, the statistical methods explored in this paper can be generalized across other tasks with discrete action space.

In the following section, we will discuss statistical and model-based approaches to determine if the frame is critical. Later, we also delineate the advantage of our methods over existing techniques in terms of efficiency and effectiveness.

III. Background

A. Reinforcement learning

In reinforcement learning (RL), we aim to optimize the performance of the agent interacting with an environment ($\mathcal{E}$). The agent perceives the current state of the environment at time t, s_t, and takes action a_t from the set of legal actions $\mathcal{A}$. The agent takes action by referring to the policy it has learned ($\pi : \mathcal{S} \longrightarrow \mathcal{A}$). By taking this action, the agent transitions to the next state s_{t+1}, and receives a reward r_{t+1}. The goal of the agent is to learn an optimal policy (π^*) that maximizes the cumulative reward ($\mathcal{R}_t$) obtained by the agent over time.

Deep Q Network (DQN)[25] approximates the Q-function defined by the Bellman's equation to estimate state-action value. This is further extended to incorporate two Q-Networks [26], a policy network that is regularly updated, and a target network that is used to take action. By introducing two networks, the value overestimation of the state is reduced. While these approaches trained the agent to estimate the Q-values, Dueling Network [27] aims to estimate the advantage of the action. The *advantage* of an action quantifies the importance of taking that action. Due to this reason, we employ the dueling architecture algorithm to train our surrogate model that is used to identify the critical frames.

The actor-critic network uses two networks. A critic network is used to estimate the value of the state-action pair. An actor network is used to choose an action for a given state. In Asynchronous Advantage Actor-Critic (A3C)[28] networks, multiple parallel threads are used to interact with the environment and update the shared model. As a result, this reduces the training time and generalizes the agent to the environment variations. Therefore, we use the A3C algorithm to train all our agents.

B. Adversarial Attacks

Given the dynamic nature of the environment upon which the RL agents are trained, they are easily deceived by naively crafted attacks. The attacker aims to reduce the aggregated reward collected by the agent during an episode. The attacker achieves this by tricking the agent to take a non-optimal action.

In this paper, we use three types of adversarial noise to perturb the state perceived by the agent - Random noise attack, Gradient-based attack [29], Fast Gradient Method (FGM) [14]. Random noise attack is a state-agnostic naively crafted attack. In gradient-based attack, we compute the gradient direction with respect to the value estimated and add noise in the direction of the gradient. In FGM attack, we compute the gradient of the estimated loss and add noise to maximize the loss.

C. Critical State

The effect of the adversarial attack greatly depends upon the state being attacked [20]. States where the model strongly favors a specific action over the other possible actions are called critical states. It is essential to identify the critical states for two reasons. Attacking the model while in these states can result in a sharp reduction in the agent's performance. Continuously or regularly attacking the states is not only computationally expensive but also easily detectable [20]. The attacker can choose to attack only when a critical state occurs. This makes it harder to detect the presence of the attack while making sure that a significant reduction in the model's performance is achieved.

Methods to identify the critical states are discussed in detail in the next section.

IV. Methodology

We modify the perceived state by the agent by adding an imperceptible noise. Eqn. 1 describes how the perceived states are calculated.

$$s_i' = s_i + \gamma \times \epsilon \times \delta \tag{1}$$

where, γ is the decision parameter, ϵ is small constant and δ is the perturbation calculated based methods describe in III-B. If γ is 1, the adversary attacks the frame, else it takes no action. Figure 1 depicts the overall system.

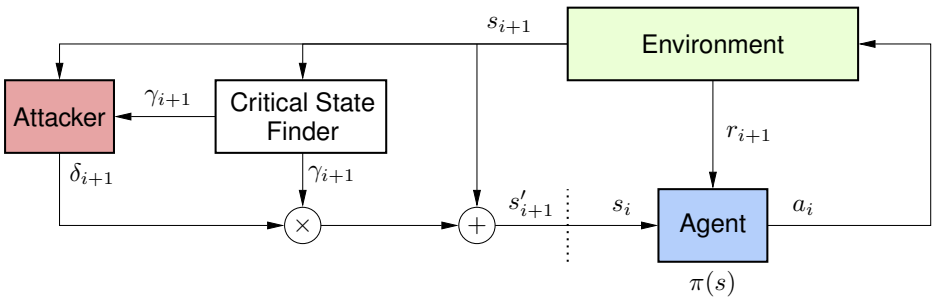

Fig. 1. Proposed system

A. Statistical Approaches

The actor network predicts the action distribution for the current state perceived by the agent$\pi(s)$. Analysis of this distribution can be used to identify critical states. We employ five methods based on a statistical analysis of the action probability distribution.

The first method (Max value method) compares the maximum value of $\pi(s)$ to a threshold value and decides if the state is critical. If the maximum value of $\pi(s)$ exceeds a set threshold β, then the state is considered critical. This method is depicted in Eqn. 2.

$$\gamma = \begin{cases} 1 & if \quad \max(\pi(s)) > \beta_1 \\ 0 & otherwise \end{cases} \tag{2}$$

Intuitively speaking, when the action probability is high enough, the agent prefers to take the action strongly. Attacking such states might mislead the agent during the crucial stages of the game. This method, however, is not ideal since it does not make use of the other facets of the distribution.

The Max-Min ratio method (Eqn. 3) also considers the probability of the least preferred action. The state is classified as critical when the ratio of the maximum to the minimum values of the probability distribution exceeds a threshold value.

$$\gamma = \begin{cases} 1 & if \quad \frac{\max(\pi(s))}{1+\min(\pi(s))} > \beta_2 \\ 0 & otherwise \end{cases} \tag{3}$$

A constant is added to the denominator to clip the value of the ratio within the range (0,1]. When the action space is large, it is highly unlikely for the maximum action probability to exceed a high threshold, given the multinomial nature of the distribution. Setting a low threshold can also lead to an inaccurate classification of the states. Using the maximum and minimum values of the distribution helps to alleviate this problem.

Alternatively, the difference between the maximum and the minimum value of the distribution can be used to determine if the state is critical (Eqn. 4) [30]. If the difference is greater than the threshold value, the state is classified as a critical state.

$$\gamma = \begin{cases} 1 & if \quad \max(\pi(s)) - \min(\pi(s)) > \beta_3 \\ 0 & otherwise \end{cases} \tag{4}$$

The disparity between the two terms is measured and used to classify the state in Eqn. 3 and 4. These methods rely only on the calculated disparity and do not take other features of the distribution into account. In some cases where the agent gives equal importance to two or more actions, these methods fail to identify the critical state. Thus, it is necessary to leverage the complete distribution to efficiently identify critical states. By analyzing the higher statistical moments of the distribution, it is possible to identify critical states that might not be detected using the previous methods.

When the agent emphasizes on choosing a specific action at a time step, the action distribution is spread more widely, resulting in increased variance. Standard deviation method (Eqn. 5) leverages this spread of the action probabilities to decide if the state is critical or not.

$$\gamma = \begin{cases} 1 & if \quad \sigma(\pi(s)) > \beta_4 \\ 0 & otherwise \end{cases} \tag{5}$$

When the distribution of the action probability during crucial points during the episode were further analyzed, it depicted a non-Gaussian distribution skewed toward the lower range. Exploiting this property of the distribution, we use the third moment, skewness, to find the critical states (Eqn. 6).

$$\gamma = \begin{cases} 1 & if \quad \frac{3\times(mean(\pi(s))-median(\pi(s)))}{\sigma(\pi(s))} > \beta_5 \\ 0 & otherwise \end{cases} \tag{6}$$

The relationship between the action probabilities (action probabilities are exclusive and exhaustive) is exploited in the standard deviation method and the skewness method.

B. Surrogate Dueling DQN

Dueling DQN (DDQN) [27] provides an estimate of the *advantage* of the action and the value of the state. The Q-values (Eqn. 7) can be then estimated by adding the *advantage* and the value.

$$\mathcal{Q}(s,a) = A(s,a) + \mathcal{V}(s) \tag{7}$$

While the Q-value dictates what action to take in a given time step, the *advantage* quantifies the importance of taking that action. Quantitatively, the *advantage* is the difference between the expected and estimated return for a state-action

TABLE I
AVERAGE REWARD RECEIVED BY THE AGENT*

Environment	Method	No attack	Random noise	Gradient based	FGM
Breakout	Max value	388.0	271.1 (1.07%)	242.6 (1.55%)	227.1 (0.78%)
	Max-min ratio		287.5 (0.69%)	194.5 (0.62%)	209.8 (0.61%)
	Max-min difference†		249.3 (0.49%)	212.0 (0.39%)	198.0 (0.48%)
	Standard deviation		240.3 (0.76%)	227.7 (0.36%)	221.7 (0.51%)
	Skewness		244.3 (0.76%)	225.2 (1.31%)	224.8 (0.87%)
	Surrogate DDQN		260.8 (0.45%)	232.7 (0.42%)	251.2 (0.31%)
	Long term impact classifier		226.3 (0.58%)	198.6 (1.34%)	207.8 (0.97%)
Pong	Max value	22.0	17.8 (0.33%)	16.4 (0.36%)	16.9 (0.32%)
	Max-min ratio		17.7 (0.16%)	16.6 (0.28%)	16.9 (0.32%)
	Max-min difference †		17.5 (0.02%)	16.3 (0.06%)	16.2 (0.05%)
	Standard deviation		16.8 (0.02%)	16.2 (0.04%)	16.3 (0.07%)
	Skewness		17.3 (0.39%)	16.5 (0.07%)	16.8 (0.33%)
	Surrogate DDQN		16.8 (1.09%)	16.3 (1.28%)	16.2 (0.99%)
	Long term impact classifier		16.6 (0.81%)	16.0 (0.65%)	16.0 (0.91%)
Seaquest	Max value	660.0	474.0 (1.25%)	448.0 (1.15%)	440.0 (1.31%)
	Max-min ratio		466.0 (0.92%)	398.0 (1.42%)	408.0 (1.24%)
	Max-min difference †		462.0 (0.65%)	392.0 (0.80%)	420.0 (0.85%)
	Standard deviation		430.0 (0.99%)	390.0 (0.74%)	380.0 (0.67%)
	Skewness		406.0 (0.62%)	376.0 (0.45%)	374.0 (0.49%)
	Surrogate DDQN		456.0 (0.99%)	378.0 (1.09%)	396.0 (0.79%)
	Long term impact classifier		389.0 (0.49%)	366.0 (0.36%)	376.0 (0.30%)

* The values enclosed within the brackets are the percentage of frames that were identified as critical and attacked over 10 episodes.
† Method discussed by Lin et al. [20]

pair. Intuitively, the *advantage* value quantifies the importance of the action while being opaque to the present state.

A surrogate DDQN was trained in the same environment as the target agent. The *advantage* estimated by this surrogate model is used to find the critical states. A high value of the *advantage* implies that the agent must take that action. This is depicted in Eqn. 8.

$$\gamma = \begin{cases} 1 & if \quad \max(A(s,a)) > \beta_{DDQN} \\ 0 & otherwise \end{cases} \tag{8}$$

C. Long-term impact classifier

Jainwen Sun et al. [23] performed series of adversarial actions from a time step and analyzed the state after certain time steps, with and without the presence of an adversary. If the adversarial actions were able to take the agent to a worse enough future state, then the attack strategy is employed at the current state. The method, however, is computationally expensive as it not only performs simulation for future M steps at each state but also compares various attack strategies for each of these simulations.

In our work, we model a classifier that can approximate the function performed by the aforementioned approach. The classifier model takes the action distribution predicted by the policy as the input and decides if the state is worthy of attacking. Since it is a binary classification problem, the cross-entropy loss is used to optimize the classifier. The ground truth to train the classifier is based on the simulation strategy discussed by Jainwen Sun et al. [23]. The aggregated reward after M steps from the current time step is separately calculated under two conditions - a) In all the M steps, the state perceived by the agent is perturbed by Gradient-based noise. b) No attack is performed. The attacker aims to maximize the difference (Eqn. 9) between the cumulative reward received by the agent under these conditions.

$$\delta = \sum_{i=j}^{j+M} r_i - \sum_{i=j}^{j+M} r'_i \tag{9}$$

The current state (s_j) is attacked if there is a minimum decrease of 35% in the received reward after attacking the M subsequent steps. The computation of the ground truth y_i is shown in Eqn. 10.

$$y_j = \begin{cases} 1 & \phi(s_j) > 0.35 \\ 0 & otherwise \end{cases} \tag{10}$$

$$\phi(s_j) = 1 - \frac{\sum_{i=j}^{j+M} r'_i}{\sum_{i=j}^{j+M} r_i} \tag{11}$$

s'_i and s_i are the perturbed and normal states respectively. r'_i and r_i are the rewards received by the agent for taking action after perceiving s'_i and s_i respectively.

V. EXPERIMENT

In our experiments, we consider three Atari 2600 games that are simulated using the Arcade Learning Environment - Pong, Breakout, and Seaquest. These environments have continuous state space and discrete action space. We model the agent as an LSTM based A3C [28], [31] network to learn to perform optimally in these environments. Both the actor and critic follow the same architecture except for the last layer. The input image is decomposed to its lower-dimensional features by four convolution layers. The feature vector is then passed to an LSTM layer. The intermediate representation from the LSTM layer is fed to a fully connected layer. For the actor

network, the output layer has dimensions equal to that of the action space. The critic network, which estimates the value of the state, has only one continuous-valued output. The model was optimized using the RMSProp algorithm.

The states of the environment are represented as 210×160 pixel images with a 128 color palette. Working with these frames without preprocessing can be computationally taxing. So, the raw input frame was reduced to a single-channel grayscale image. Furthermore, a window of 80×80 from the center of the frame is cropped and normalized. The model was trained on these preprocessed frames for one million time steps for each of the three environments.

Surrogate DDQN was trained in the same environments for 500,000 time steps. This model followed the same architecture discussed for the A3C. However, it optimized the estimation of Q-values as calculated in Eqn. 7.

The long-term impact classifier is a shallow fully connected network with one hidden unit of size 128×1. During training, at each time step, we choose $M = 5$ and perform a simulation of five future steps and calculate the decrease in the cumulative reward. When the reduction is greater than 35%, we set the ground truth y_i to 1, according to Eqn. 10.

VI. Results

The performance of the agent is measured by calculating the average reward received by the agent over 10 episodes. Table I shows the performance of the A3C agent under normal and adversarial conditions. Furthermore, the ratio of the frames that were detected and attacked to the total number of frames in each of the methods is also indicated.

We observe a significant decrease in the average reward collected by the agent even when less than 1% of the frames were attacked. On an average, we notice a performance drop of 43.54%, 25.78%, and 40.52% in Breakout, Pong, and Seaquest environments, respectively.

It is important to note that Pong is comparatively a simple environment. Therefore, it is relatively hard to attack the agent by deceiving it to take a sub-optimal action. Furthermore, the A3C network is trained using samples collected from multiple threads and updated asynchronously. Therefore, it is generalized to input variation and hence, has some degree of robustness.

Across the types of adversarial noise crafted, we observe that gradient based noise was able to achieve maximum performance degradation while attacking a relatively less number of frames than the other types of noise. This is because the gradient based noise is computed based on the gradient of the input with respect to the output [29]. We craft a noise that is in the opposite direction of the gradient to deceive the agent to take a sub-optimal action. Random noise is a naively crafted attack that does not guarantee a performance decline. Thus, we notice that it had minimal effect on the agent's performance.

When the threshold β was adjusted to allow the adversary to attack more states, we did not notice an appreciable drop in performance. In the case of Seaquest, the skewness threshold was reduced to allow perturbation of 5.27% of the total number of frames (which is 15 times the number of frames chosen to attack previously). However, the further drop in performance was only by 10%. This experiment further corroborates that our method has accurately identified most of the critical frames. As the ratio of frames that are attacked is increased, we notice that the agent's performance gradually declined. Eventually, the average reward saturated to 0 for Breakout and Seaquest, and at -20 for Pong. However, this increased the average time taken for completion of the episode by nine folds (from 60.3 seconds to 536.7 seconds).

Across all the environments, we observe that there is a maximum average reward decline in the case of long-term impact classifier. The critical point attack method discussed by Sun et al. [23] searched for an optimal adversarial action sequence of M steps at each time step. However, this is computationally inefficient and cannot be used to identify critical frames for real-time applications. The model uses GPU to perform the prediction, thus reducing the computational overhead on the CPU. On the other hand, the state transitions can only be computed using the CPU, making the simulation less realisable for real-time applications. Hence, our long-term impact classifier is computationally more efficient. Our experiments show that our method (98.2 seconds) took 80.3% lesser compute time than the critical point method (577.6 seconds) discussed in [23], when M was set to 5.

VII. Conclusion

In this paper, we show that the performance of a DRL agent can be drastically reduced by attacking less than 1% of the total number of states. We also introduce a classifier approach that reduces the computational overhead while maintaining the effectiveness of the method. Our methods identifies the critical frames using information that is available in a black box setting (no access to model parameters). We further show that, increase in the number of frames correlated to a gradual decline in the agent's performance. However, the average compute time was increased by 900%. In the future, more efficient attack strategies could be employed to further reduce the agent's performance by attacking fewer states.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012. [Online]. Available: https://proceedings.neurips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

[2] A. Y. Hannun, C. Case, J. Casper, B. Catanzaro, G. Diamos, E. Elsen, R. Prenger, S. Satheesh, S. Sengupta, A. Coates, and A. Y. Ng, "Deep speech: Scaling up end-to-end speech recognition," *CoRR*, vol. abs/1412.5567, 2014. [Online]. Available: http://arxiv.org/abs/1412.5567

[3] A. Torfi, R. A. Shirvani, Y. Keneshloo, N. Tavaf, and E. A. Fox, "Natural language processing advancements by deep learning: A survey," *CoRR*, vol. abs/2003.01200, 2020. [Online]. Available: https://arxiv.org/abs/2003.01200

[4] H. Fujiyoshi, T. Hirakawa, and T. Yamashita, "Deep learning-based image recognition for autonomous driving," *IATSS Research*, vol. 43, no. 4, pp. 244–252, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0386111219301566

[5] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.

[6] S. D. Holcomb, W. K. Porter, S. V. Ault, G. Mao, and J. Wang, "Overview on deepmind and its alphago zero ai," in *Proceedings of the 2018 International Conference on Big Data and Education*, ser. ICBDE '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 67–71. [Online]. Available: https://doi.org/10.1145/3206157.3206174

[7] A. P. Badia, B. Piot, S. Kapturowski, P. Sprechmann, A. Vitvitskyi, Z. D. Guo, and C. Blundell, "Agent57: Outperforming the atari human benchmark," in *International Conference on Machine Learning*. PMLR, 2020, pp. 507–517.

[8] S. Huang, N. Papernot, I. Goodfellow, Y. Duan, and P. Abbeel, "Adversarial attacks on neural network policies," 2017.

[9] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," 2014.

[10] X. Yang, W. Liu, S. Zhang, W. Liu, and D. Tao, "Targeted attention attack on deep learning models in road sign recognition," *IEEE Internet of Things Journal*, vol. 8, no. 6, pp. 4980–4990, 2021.

[11] A. Chernikova, A. Oprea, C. Nita-Rotaru, and B. Kim, "Are self-driving cars secure? evasion attacks against deep neural networks for steering angle prediction," in *2019 IEEE Security and Privacy Workshops (SPW)*. IEEE, 2019, pp. 132–137.

[12] Y. Zhao, I. Shumailov, H. Cui, X. Gao, R. Mullins, and R. Anderson, "Blackbox attacks on reinforcement learning agents using approximated temporal information," 2019.

[13] A. Nguyen, J. Yosinski, and J. Clune, "Deep neural networks are easily fooled: High confidence predictions for unrecognizable images," 2015.

[14] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2015.

[15] D. Wu, Y. Wang, S.-T. Xia, J. Bailey, and X. Ma, "Skip connections matter: On the transferability of adversarial examples generated with resnets," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=BJlRs34Fvr

[16] F. Tramèr, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, "The space of transferable adversarial examples," 2017.

[17] X. Tan, C.-B. Chng, Y. Su, K.-B. Lim, and C.-K. Chui, "Robot-assisted training in laparoscopy using deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 485–492, 2019.

[18] A. Gleave, M. Dennis, C. Wild, N. Kant, S. Levine, and S. Russell, "Adversarial policies: Attacking deep reinforcement learning," 2021.

[19] T. Chen, J. Liu, Y. Xiang, W. Niu, E. Tong, and Z. Han, "Adversarial attack and defense in reinforcement learning-from ai security view," *Cybersecurity*, vol. 2, 12 2019.

[20] Y.-C. Lin, Z.-W. Hong, Y.-H. Liao, M.-L. Shih, M.-Y. Liu, and M. Sun, "Tactics of adversarial attack on deep reinforcement learning agents," in *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, ser. IJCAI'17. AAAI Press, 2017, p. 3756–3762.

[21] Y. Qiaoben, X. Zhou, C. Ying, and J. Zhu, "Strategically-timed state-observation attacks on deep reinforcement learning agents," in *ICML 2021 Workshop on Adversarial Machine Learning*, 2021. [Online]. Available: https://openreview.net/forum?id=FSD_8Sglf_u

[22] C.-H. H. Yang, J. Qi, P.-Y. Chen, Y. Ouyang, I.-T. D. Hung, C.-H. Lee, and X. Ma, "Enhanced adversarial strategically-timed attacks against deep reinforcement learning," in *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020, pp. 3407–3411.

[23] J. Sun, T. Zhang, X. Xie, L. Ma, Y. Zheng, K. Chen, and Y. Liu, "Stealthy and efficient adversarial attacks against deep reinforcement learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, pp. 5883–5891, Apr. 2020. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/6047

[24] Q. Xinghua, Z. Sun, Y. Ong, A. Gupta, and P. Wei, "Minimalistic attacks: How little it takes to fool deep reinforcement learning policies," *IEEE Transactions on Cognitive and Developmental Systems*, vol. PP, pp. 1–1, 02 2020.

[25] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," 2013.

[26] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," 2015.

[27] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," 2016.

[28] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," 2016.

[29] A. Pattanaik, Z. Tang, S. Liu, G. Bommannan, and G. Chowdhary, "Robust deep reinforcement learning with adversarial attacks," 2017.

[30] Y.-C. Lin, Z.-W. Hong, Y.-H. Liao, M.-L. Shih, M.-Y. Liu, and M. Sun, "Tactics of adversarial attack on deep reinforcement learning agents," in *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI-17*, 2017, pp. 3756–3762. [Online]. Available: https://doi.org/10.24963/ijcai.2017/525

[31] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.